

Nota PSR N1-N2 ↔ Camada de Planejamento Implementada

Mapeamento termo-a-termo entre “Reforços interconexões N1-N2 através de equilíbrios de Stackelberg e Nash” (PSR, jun. 2026) e `src/planning/`

Trabalho derivado da leitura direta de `docs/references/Expansão-N1-N2-EquilíbriosStackelberg-Nash-v2 (1).pdf` (nota interna PSR, 9 pp., jun. 2026) e da implementação em Julia do módulo `src/planning/` do projeto TSO-DSO (`master.jl`, `follower.jl`, `subproblem.jl`, `benders.jl`, `coupling.jl`, `nash.jl`), resolvida via decomposição de Benders + diagonalização de Gauss-Seidel.

1 Resumo

Este documento é o **complemento**, não o substituto, de `modelo_stackelberg_dso_unico.typ` (que já narra a implementação da camada de planejamento em prosa livre). Aqui o objetivo é mais estrito: para **cada** equação da nota PSR — problemas (1), (2), (4), (7) e a extensão inteira (8)-(9), com seus rótulos (1a)-(1j), (2a)-(2e), (4a)-(4f) — apontar o construtor Julia específico (arquivo + nome da variável/restrição) que a realiza, ou declarar explicitamente que ela ainda não foi implementada. Nenhuma equação da nota é omitida silenciosamente: cada uma recebe uma etiqueta explícita **Equivalente**, **Desvio deliberado** ou **Não implementado**.

A nota PSR foi lida diretamente do PDF primário (não apenas do dígist condensado em `.planning/research/THEORY-papers.md`) — ver a nota de leitura na Seção 3 sobre uma discrepância encontrada entre o dígist e a fonte primária quanto aos rótulos N1/N2.

2 Conjuntos e índices

Nota PSR	Significado (PSR)	Contraparte no repositório
$s \in S$	índice de cenário (a nota pondera por $\frac{1}{S}$ em toda parte)	Não implementado — o repositório não tem índice de cenário; cada iteração de Benders (<code>solve_stackelberg!</code>) resolve um trial determinístico único z_k , não uma expectativa $\left(\frac{1}{S}\right) \sum_s (\cdot)$. Ver INT-STRETCH/análise de estocasticidade em <code>STATE.md</code> (fora do escopo desta nota).
N1	sistema de transmissão (variáveis x)	<code>follower.jl</code> — <code>FollowerLP(x_inv, x_op[t])</code>
N2	distribuidora (variáveis y, z)	<code>master.jl</code> — <code>BendersMaster(y_inv, z[t])</code> + <code>subproblem.jl</code> — <code>PlanningOracle</code> (a operação de N2)
primeiro nível / líder	problema (4), a distribuidora	<code>master.jl</code> — <code>BendersMaster</code> , resolvido por <code>solve_master!</code>
segundo nível / seguidor	problema (2), o sistema de transmissão	<code>follower.jl</code> — <code>FollowerLP</code> , resolvido por <code>solve_follower!</code>

—	não há um terceiro nível explícito na nota — a operação de N2 (eq. 1f) entra linearmente no objetivo do líder	<code>subproblem.jl</code> — <code>PlanningOracle</code> : um SOCP de bem-estar completo (tese eq. 3.31–3.45) que a nota PSR trata apenas como um balanço linear abstrato; o repositório precisa de um terceiro subproblema porque essa operação deixou de ser linear
---	---	--

3 Estrutura do jogo

3.1 Nota de leitura: os rótulos N1/N2 e líder/seguidor

O parágrafo introdutório da nota PSR (p. 3) é explícito: “...reforços nas interligações para importação de energia das distribuidoras (N2) do sistema de transmissão (N1)...” — ou seja, **N1 = sistema de transmissão**, **N2 = distribuidora**. Essa correspondência é usada de forma consistente em todo o documento: “(1a) e (1d) representam restrições de investimentos em N1 e N2, respectivamente” (as variáveis x ficam com N1; as variáveis y com N2), e mais adiante, de forma explícita: “*Problema (4) pode ser interpretado como o problema de líder ou de primeiro nível de Stackelberg e o problema (2) como o problema de segundo nível.*” Como o problema (4) é o problema- y (distribuidora/N2) e o problema (2) é o problema- x (transmissão/N1), a nota primária é **internamente consistente**: distribuidora (N2, y) = líder = primeiro nível; transmissão (N1, x) = seguidor = segundo nível.

Achado de leitura: o dígest em `.planning/research/THEORY-papers.md` (linha 29) parafraseia essa correspondência **invertida** (“N1 = distribution side; N2 = transmission side”) — uma transcrição equivocada do dígest, não uma ambiguidade genuína da nota primária. A correspondência de variáveis usada pelo próprio dígest ($x_{\text{inv}}, x_{\text{op},s} \leftrightarrow \text{N1/transmissão}$; $y_{\text{inv}}, y_{\text{op},s} \leftrightarrow \text{N2/distribuidora}$) já está correta; apenas a frase de abertura do dígest inverteu os rótulos textuais N1/N2. Este documento adota a leitura direta da fonte primária — a MESMA leitura que a implementação já assume: `master.jl` (variáveis y) = líder = distribuidora; `follower.jl` (variáveis x) = seguidor = transmissão.

3.2 Uma distribuidora → Stackelberg

Problema (4) (líder, distribuidora/N2) vs. problema (2) (seguidor, transmissão/N1) → `solve_stackelberg!` (`benders.jl`): a cada iteração k , resolve o mestre (`solve_master!`), depois o seguidor no trial z_k (`solve_follower!`), depois — se o seguidor for factível — o oráculo operacional (`solve_planning_oracle!`), acrescentando cortes de otimalidade ao mestre. Ver Seção 7 para a derivação completa do laço.

3.3 Múltiplos distribuidores → Nash via diagonalização de Gauss-Seidel

Problema (7) (equilíbrio de Nash, I distribuidoras) → `coupling.jl` (`SharedTransmission`, o corredor de transmissão compartilhado) + `nash.jl` (`run_nash!`, o laço externo de diagonalização de Gauss-Seidel). Ver Seção 8.

4 Variáveis — mapeamento lado a lado

Símbolo PSR	Significado (PSR)	Identificador Julia
x_{inv}	investimento em reforço de N1 (transmissão)	<code>follower.jl</code> — <code>FollowerLP.x_inv</code> (contínuo, $0 \leq x_{\text{inv}} \leq x_{\text{inv_max}}$)
$x_{\text{op},s}$	operação de N1 no cenário s	<code>follower.jl</code> — <code>FollowerLP.x_op[t]</code> (sem índice de cenário — t é hora, não cenário)
y_{inv}	investimento genérico de N2 (distribuidora)	<code>master.jl</code> — <code>BendersMaster.y_inv</code> (contínuo, $0 \leq y_{\text{inv}} \leq y_{\text{max}}$)

$y_{op,s}$	operação genérica de N2 no cenário s (balanço linear abstrato, eq. 1f)	subproblem.jl — PlanningOracle: TODA a rede branch-flow SOCP + dispositivos dos agregadores (tese eq. 3.31–3.45) — a operação de N2 real, não abstrata
$y_{inv,flex}$	investimento especificamente em flexibilidade (baterias, agregadores)	Colapsado em master.jl's y_{inv} — o repositório não distingue investimento “genérico” de N2 de investimento “em flexibilidade”; y_{inv} é o investimento em flexibilidade
$z_{flex,s}$	operação de flexibilidade (fluxo habilitado pela flexibilidade)	master.jl — BendersMaster.z[t], limitado por box_hi[t]: $z[t] \leq y_{inv}$
$z_{x,s}$	fluxo de exportação de N1 na interligação, cenário s	follower.jl — FollowerLP.z[t] (Parameter, ajustado via set_parameter_value. a cada trial de Benders)
$z_{y,s}$	fluxo de importação de N2 na interligação, cenário s	master.jl — BendersMaster.z[t] (a MESMA variável física que subproblem.jl's $p_{import}[t]$, via pin[t]: $p_{import}[t] == z[t]$)
π_s	dual do acoplamento (2e) — custo marginal de reforço	follower.jl — $\pi_s = \text{dual}(\text{f.coupling})$, retornado por solve_follower!
w^k	valor ótimo do problema (2) no trial z^k — o termo constante do corte	benders.jl — follower_res.cost (passado como cost_k a add_optimality_cut!(master, :x, ...))

5 Objetivo — comparação

PSR problema (1), integrado (uma distribuidora):

$$\text{Min} \left\{ c_{x,inv} x_{inv} + c_{y,inv} y_{inv} + c_{y,inv,flex} y_{inv,flex} + \frac{1}{S} \sum_s (c_{x,op} x_{op,s} + c_{y,op} y_{op,s}) \right\} \quad (1)$$

PSR problema (2), seguidor (N1/transmissão), parametrizado por $z_{y,s}$:

$$\alpha(\{z_{y,s}\}) = \text{Min} \left\{ c_{x,inv} x_{inv} + \frac{1}{S} \sum_s c_{x,op} x_{op,s} \right\} \quad (2)$$

PSR problema (4), líder (N2/distribuidora), com cortes de Benders:

$$\text{Min} \left\{ c_{y,inv} y_{inv} + c_{y,inv,flex} y_{inv,flex} + \frac{1}{S} \sum_s c_{y,op} y_{op,s} + \alpha \right\} \quad (4)$$

Repositório — follower.jl (segue exatamente a forma de (2)):

$$\alpha_{x(z)} = \text{Min}_{x_{inv}, x_{op}} \left\{ c_{inv} \cdot x_{inv} + \sum_t c_{op[t]} \cdot x_{op[t]} \right\}$$

Repositório — master.jl (o mestre do líder):

$$\text{Min}_{y_{inv}, z, \alpha_{op}, \alpha_x} \{ c_y \cdot y_{inv} + \alpha_{op} + \alpha_x \}$$

Repositório — subproblem.jl (o oráculo — bem-estar como custo negativo):

$$\alpha_{op}(z) = -\max \left[\sum_j U_{ag_j} - \sum_t \lambda_0[t] \cdot p_{import[t]} \right]$$

Comparação, termo a termo: $c_{y,inv} y_{inv} + c_{y,inv,flex} y_{inv,flex}$ (PSR, dois termos de investimento) → master.jl's único termo $c_y \cdot y_{inv}$ — o repositório colapsa os dois investimentos de N2 da nota em um só

(**Equivalente**, com o colapso de nomenclatura já documentado em Seção 4). O termo $(\frac{1}{S}) \sum_s c_{y,op} y_{op,s}$ (PSR, custo **linear** abstrato de operação de N2) → o repositório substitui esse termo pelo **negativo do ótimo de bem-estar** de um SOCP genuíno ($\alpha_{op}(z)$), não uma expressão linear — **Equivalente (reforçado)**: o repositório precisa de um epígrafe de Benders para esse termo justamente porque ele deixou de ser linear; a nota PSR não precisa, porque trata $y_{op,s}$ como linear e o inclui diretamente no objetivo do líder. O termo α (PSR, único epígrafe cobrindo apenas o custo de reforço de N1) → `master.jl`'s α_x — mapeamento direto (**Equivalente**). O repositório introduz um SEGUNDO epígrafe (α_{op}) que a nota PSR não tem — ver Seção 7 para a justificativa completa.

6 Restrições — derivação constrangimento-a-constrangimento

6.1 Problema (1) — formulação integrada, uma distribuidora

Eq.	Significado (PSR)	Construtor Julia	Etiqueta
(1a)	$Ax_{inv} \leq b$ — restrições genéricas de investimento em N1 (transmissão).	<code>follower.jl: 0 <= x_inv <= x_inv_max (build_follower).</code>	Equivalente
(1b)	$x_{op,s} \leq M_x x_{inv}$ — capacidade operacional limitada pelo investimento em N1.	<code>follower.jl: invest_op[t]: x_op[t] <= corridor_cap * x_inv.</code>	Equivalente
(1c)	$F_x x_{op,s} = d_{x,s}$ — balanço de demanda na rede interna de N1, por cenário.	Nenhum — <code>follower.jl</code> não modela uma rede de transmissão interna, apenas um corredor escalar agregado.	Desvio deliberado
(1d)	$By_{inv} \leq h$ — restrições genéricas de investimento em N2 (distribuidora).	<code>master.jl: 0 <= y_inv <= y_max (build_master).</code>	Equivalente
(1e)	$y_{op,s} \leq M_y y_{inv}$ — operação de N2 limitada pelo investimento.	<code>master.jl: box_hi[t]: z[t] <= y_inv.</code>	Equivalente
(1f)	$F_y y_{op,s} = d_{y,s}$ — balanço de demanda na rede interna de N2 (a operação real da distribuidora).	<code>subproblem.jl: balance_p[j,t]/balance_q[j,t]</code> — a rede branch-flow SOCP COMPLETA da tese (eq. 3.31–3.45), com dispositivos dos agregadores.	Equivalente (reforçado)
(1g)	$z_{x,s} - H_x x_{op,s} = 0$ — balanço de fronteira em N1.	<code>follower.jl: coupling[t]: x_op[t] == z[t]</code> (H_x especializado à identidade).	Equivalente
(1h)	$z_{y,s} - H_y y_{op,s} - H_{y,flex} z_{flex,s} = 0$ — balanço de fronteira em N2 + contribuição de flexibilidade.	<code>subproblem.jl: pin[t]: p_import[t] == z[t]</code> , combinado com o balanço nodal na barra-raiz (<code>balance_p[feeder.root,t]</code> , que já soma todas as contribuições dos agregadores/flexibilidade).	Equivalente (embutido no balanço de rede, não uma linha H_y separada)
(1i)	$z_{flex,s} \leq H_{y,invflex} y_{inv,flex}$ — operação de flexibilidade limitada pelo investimento em flexibilidade.	<code>master.jl: a MESMA linha box_hi[t]: z[t] <= y_inv</code> que realiza (1e) — no repositório, $z[t]$ desempenha tanto o papel de $y_{op,s}$ quanto o de $z_{flex,s}$, já que a única operação de N2 modelada no nível do líder É a flexibilidade.	Equivalente ($H_{y,invflex}$ especializado à identidade)
(1j)	$z_{x,s} - z_{y,s} = 0$ — acoplamento N1↔N2 (exportação = importação), a ÚNICA restrição que monta o acoplamento explicitamente.	Nenhuma restrição explícita — <code>follower.jl</code> e <code>subproblem.jl</code> referenciam a MESMA variável $z[t]$ (o z do mestre, passado como <code>z_trial</code> a ambos os subproblemas a cada iteração de Benders).	Equivalente (por construção) — o repositório colapsa $z_{x,s}/z_{y,s}$ em uma única variável compartilhada, em vez de duas variáveis ligadas por uma igualdade

6.2 Problema (2) — subproblema do seguidor (N1/transmissão), parametrizado por $z_{y,s}$

Eq.	Significado (PSR)	Construtor Julia	Etiqueta
(2a)	$Ax_{\text{inv}} \leq b.$	follower.jl: $0 \leq x_{\text{inv}} \leq x_{\text{inv_max}}.$	Equivalente
(2b)	$x_{\text{op},s} \leq M_x x_{\text{inv}}.$	follower.jl: $\text{invest_op}[t].$	Equivalente
(2c)	$F_x x_{\text{op},s} = d_{x,s}.$	Nenhum — sem rede interna de N1 (mesma lacuna de (1c)).	Desvio deliberado
(2d)	$z_{x,s} - H_x x_{\text{op},s} = 0.$	follower.jl: $\text{coupling}[t]: x_{\text{op}}[t] == z[t].$	Equivalente
(2e)	$\frac{1}{S} z_{x,s} = \frac{1}{S} z_{y,s},$ dual π_s — o acoplamento cujo multiplicador é o custo marginal de reforço.	follower.jl: o MESMO $\text{coupling}[t]$, cujo dual é lido em <code>solve_follower!</code> como $\pi_s = \text{dual}(\text{f.coupling})$ — exatamente o π_s da nota.	Equivalente na forma do multiplicador; Desvio deliberado no fator de escala — sem índice de cenário s , sem a média $(\frac{1}{S}) \sum_s$: cada iteração de Benders é um trial determinístico único, não uma expectativa

6.3 Problema (4) — mestre do líder (N2/distribuidora), com cortes de Benders

Eq.	Significado (PSR)	Construtor Julia	Etiqueta
(4a)	$By_{\text{inv}} \leq h.$	master.jl: $0 \leq y_{\text{inv}} \leq y_{\text{max}}.$	Equivalente
(4b)	$y_{\text{op},s} \leq M_y y_{\text{inv}}.$	master.jl: $\text{box_hi}[t]: z[t] \leq y_{\text{inv}}.$	Equivalente
(4c)	$F_y y_{\text{op},s} = d_{y,s}.$	subproblem.jl: $\text{balance_p}/\text{balance_q}$ (a rede branch-flow completa) — a mesma correspondência reforçada de (1f).	Equivalente (reforçado)
(4d)	$z_{y,s} - H_y y_{\text{op},s} - H_{y,\text{flex}} z_{\text{flex},s} = 0.$	subproblem.jl: $\text{pin}[t] + \text{balanço nodal na raiz}$ — mesma correspondência de (1h).	Equivalente (embutido)
(4e)	$z_{\text{flex},s} \leq H_{y,\text{invflex}} y_{\text{inv,flex}}.$	master.jl: $\text{box_hi}[t]$ — mesma correspondência de (1i).	Equivalente
(4f)	$\alpha \geq w^k + \frac{1}{S} \sum_s \pi_s^k (z_{y,s} - z_{y,s}^k), \quad k = 1, \dots, K$ — os CORTES DE BENDERS.	master.jl: $\text{add_optimality_cut!}$ — $\alpha \geq \text{cost_k} + \sum_t \text{grad_k}[t] * (z[t] - z_k[t])$, forma matematicamente idêntica.	Equivalente na forma; Desvio deliberado na cardinalidade — ver Seção 7

6.4 Extensão inteira — problemas (8)-(9)

A nota PSR trata o caso em que $x_{\text{inv}}, x_{\text{op},s}$ (e o fluxo de interligação, via expansão binária) são variáveis inteiras (Seção 4), formando um MIP (problema 8) resolvido por relaxação Lagrangeana das restrições de cópia (problema 9), com multiplicadores obtidos por maximização do Lagrangeano — a máquina de Stochastic Dual Dynamic Integer Programming / cortes L-shaped inteiros (Zou-Ahmed-Sun 2019; Bansal-Küçükyavuz 2025).

Esta extensão está IMPLEMENTADA no repositório desde a Fase 24 (v3.0), estendida ao equilíbrio de Nash multi-distribuidor $N > 1$ na Fase 31 (v4.0, Seção 8) — mas por um caminho deliberadamente distinto do da nota em dois pontos: (i) a variável integerizada é o investimento do LÍDER y_{inv} (N2, distribuidora), via `master_integer.jl's build_master_integer`, não o investimento/operação/fluxo do SEGUIDOR $x_{\text{inv}}, x_{\text{op},s}$ (N1, transmissão) que a nota tem como alvo; (ii) o recourse é imposto por cortes L-shaped

inteiros de Laporte-Louveaux (`add_ll_cut!`), não pela relaxação Lagrangeana de restrições de cópia da nota (problema 9).

Eq.	Conteúdo (PSR)	Etiqueta
(8a)–(8c)	Restrições de investimento/operação/balço de N1, herdadas de (1a)–(1c), agora com x_{inv} , $x_{\text{op},s}$ possivelmente inteiros.	Desvio deliberado — <code>follower.jl</code> permanece contínuo; o repositório <code>integeriza</code> y_{inv} (N2/líder), não x_{inv} , $x_{\text{op},s}$ (N1/seguidor) — ver <code>master_integer.jl</code> 's <code>BendersMasterInteger</code>
(8d)	$z_{x,s} - H_x x_{\text{op},s} = 0$ (mesma forma de (1g)/(2d)).	Não implementado — sem expansão binária do lado de N1 (mesma lacuna estrutural de (1c)/(2c); fora do escopo da extensão inteira do repositório, que mira y_{inv})
(8e)	Expansão binária do fluxo de interligação: $z_{x,s} = \Delta \sum_j u_{j,y,s}^+ 2^j - \Delta \sum_j u_{j,y,s}^- 2^j$.	Implementado (Fase 24/31), desvio deliberado no alvo — <code>build_master_integer's</code> <code>y_inv = (y_max\2^K) sum_k 2^(k-1) b_k</code> é a expansão binária ANÁLOGA do investimento y_{inv} (D-01/D-02), não do fluxo $z_{x,s}$
(8f)–(8g)	Restrições de cópia $\left(\frac{1}{S}\right) u_{j,y,s}^+ = \left(\frac{1}{S}\right) n_{j,y,s}^+$ / $\left(\frac{1}{S}\right) u_{j,y,s}^- = \left(\frac{1}{S}\right) n_{j,y,s}^-$, cujos duais $\pi_{j,y,s}^+/\pi_{j,y,s}^-$ formam os cortes Lagrangeanos.	Desvio deliberado — nenhuma restrição de cópia explícita; <code>add_ll_cut!</code> (<code>master_integer.jl</code>) acrescenta diretamente um corte de Laporte-Louveaux sobre <code>master.b</code> , $\theta \geq (Q_\nu - L) \cdot D(b) + L$ — um plano de corte exato por canto visitado, não uma relaxação dual
(9a)–(9e)	O Lagrangeano relaxado do problema (8) e a maximização de seus multiplicadores ótimos.	Desvio deliberado — nenhum Lagrangeano é relaxado/maximizado; <code>benders.jl</code> 's <code>corner_recourse/ll_cut_recourse</code> computam Q_ν EXATAMENTE via busca ternária de cantos e <code>add_ll_cut!</code> acrescenta o corte diretamente

Desvio deliberado (mecanismo de recourse): o repositório usa cortes L-shaped inteiros de Laporte-Louveaux (Laporte & Louveaux 1993; Birge & Louveaux 2011, Seção 5.2 — “Binary First-Stage Variables”) para impor o valor exato do recourse $Q(b^\nu)$ em cada canto binário visitado (`add_ll_cut!`, `master_integer.jl`), nunca a relaxação Lagrangeana das restrições de cópia que a nota PSR usa (problema 9, Zou-Ahmed-Sun 2019). Ambos endereçam o mesmo problema geral — um MIP com recourse convexo caro de reavaliar a cada candidato — por caminhos matematicamente distintos: um plano de corte exato por canto visitado vs. uma relaxação dual com multiplicadores maximizados.

`master_integer.jl`'s `build_master_integer` cria exatamente as variáveis binárias/inteiras que a seção anterior da nota tratava como ausentes: `b[1:K]` (`Vector{VariableRef}`, binárias, $K=4$ por padrão) são as ÚNICAS variáveis binárias/inteiras de `src/planning/` hoje — `y_inv` é uma expressão DERIVADA (`AffExpr`) sobre elas, nunca uma variável própria nessa formulação (D-01/D-02). O corte de `recourse` (`add_ll_cut!`) e a detecção de ciclo/anti-stall (`apply_integer_cuts!`, `master.visited`) operam diretamente sobre `b`, nunca sobre `y_inv`. Implementado desde a Fase 24 (v3.0) e, desde a Fase 31 (v4.0), também disponível no equilíbrio de Nash multi-distribuidor $N>1$ via `run_nash!`'s kwarg `integer` (ver Seção 8) — regressão-testado em `test/test_planning_master_integer.jl` (construção/validação de limites) e `test/test_planning_nash_integer.jl` ($N=2$, $K=4$, certificação por força bruta).

7 Cortes de Benders — derivação

A nota PSR usa um ÚNICO epígrafe α (4f) porque o custo de operação de N2, $y_{op,s}$, é **linear** e entra diretamente no objetivo do líder (eq. 4) — não precisa de sua própria função de valor convexa/epígrafe, apenas o custo de reforço de N1 (função de z via o problema 2) precisa.

O repositório precisa de DOIS epígrafes (`master.jl`'s α_{op} , α_x) porque sua operação de N2 (`subproblem.jl`'s `PlanningOracle`) É um SOCP de bem-estar genuíno — não uma expressão linear substituível diretamente no objetivo do líder. `benders.jl`'s `add_optimality_cut!(master, :op, -oracle_res.cost, oracle_res. $\pi$ , lb_res.z)` e `add_optimality_cut!(master, :x, follower_res.cost, follower_res. $\pi_s$ , lb_res.z)` geram, a cada iteração factível, um corte por subproblema — Benders **multi-corte** — na MESMA forma matemática de (4f):

$$\alpha \geq \text{cost}_k + \sum_t \text{grad}_k[t] \cdot (z[t] - z_k[t])$$

Além disso, o repositório tem um ramo que a nota PSR não especifica explicitamente: cortes de FACTIBILIDADE, quando o seguidor (`follower.jl`) é infactível no trial z_k — um certificado de Farkas genuíno do HiGHS (`dual_status(model) == MOI.INFEASIBILITY_CERTIFICATE`), nunca uma heurística de “sempre factível”:

$$v_k + \sum_t u_k[t] \cdot (z[t] - z_k[t]) \leq 0$$

Isso é um **acréscimo deliberado** de robustez sobre a nota PSR (que assume implicitamente a factibilidade do problema 2 em todo o texto) — não uma correspondência com nenhuma equação numerada da nota, mas uma extensão consistente com sua própria intenção de decomposição de Benders.

8 Equilíbrio de Nash multi-distribuidor

Problema (7) da nota PSR: para I distribuidoras, cada uma resolve seu próprio problema (6) (a distribuidora i como função das injeções fixas das outras, $\{z_{j,y,s}, j \neq i\}$), e o equilíbrio de Nash é o conjunto de decisões em que nenhuma distribuidora pode melhorar sua função de custo dado o que as outras fazem — “calculado através de um processo de diagonalização em que cada distribuidora é otimizada fixando os fluxos das outras distribuidoras calculadas na última vez que foram otimizadas” (nota PSR, p. 8).

Eq.	Significado (PSR)	Construtor Julia	Etiqueta
(7a)–(7e)	Restrições de investimento/operação/acoplamento/flexibilidade da distribuidora i (mesma forma de (6a)–(6e), herdadas de (1d)–(1i)).	<code>coupling.jl</code> : <code>SharedTransmission</code> — <code>x_inv[i]</code> , <code>x_op[i,t]</code> , <code>coupling[i,t]</code> : <code>x_op[i,t] == z[i,t]</code> (uma linha independentemente dualizável por distribuidor por hora).	Equivalente

(7f)	$\alpha_i \geq w_i^k (\{z_{j,y,s}^k, j \neq i\}) + \frac{1}{S} \sum_s \pi_{i,s}^k (\{z_{j,y,s}^k, j \neq i\}) (z_{i,y,s}^k - z_{i,y,s}^k)$ — os cortes de Benders da distribuidora i , dependentes dos fluxos das outras.	nash.jl: <code>run_nash!</code> chama <code>solve_stackelberg!</code> (uma resolução COMPLETA do Benders de (4f), fresca a cada resposta ótima) com um <code>DistributorView(shared, i)</code> — o mestre de cada distribuidor i acumula seus PRÓPRIOS cortes, começando vazio a cada resposta ótima (decisão travada de correção, <code>coupling.jl</code>).	Equivalente
(7) — o equilíbrio	$\{(y_{i,inv}^e, y_{i,op,s}^e, z_{i,y,s}^e, \alpha_{i,s}^e), i = 1, \dots, I\}$ tal que cada i resolve seu próprio problema ótimo dado $\{z_{j,y,s}^e, j \neq i\}$.	nash.jl: <code>run_nash!</code> — laço externo de Gauss-Seidel: para cada distribuidor i em ordem, <code>activate_distributor!</code> → <code>solve_stackelberg!</code> → <code>write_back!</code> IMEDIATAMENTE (nunca Jacobi) → repete até <code>is_converged(trace, tol_outer, N)</code> .	Equivalente (estrutura de Gauss-Seidel idêntica)

Desvio deliberado (alocação de custo de investimento compartilhado): a nota PSR não especifica como o custo de um reforço de transmissão COMPARTILHADO por múltiplas distribuidoras deveria ser dividido entre elas — o problema (6)/(7) trata cada y_i/x com uma notação que sugere um x potencialmente único por par de distribuidoras, sem resolver a questão de alocação de custo explicitamente. `coupling.jl` resolve essa ambiguidade adotando **propriedade de investimento por distribuidor**: cada distribuidor i possui seu próprio $x_{inv[i]}$ e paga seu próprio $c_{inv[i]} \cdot x_{inv[i]}$; o objeto genuinamente COMPARTILHADO é apenas a capacidade agregada do corredor (`capacity[t]`: $\sum_i x_{op}[i,t] \leq \text{corridor_cap} * \sum_i x_{inv}[i]$) — uma decisão de modelagem documentada explicitamente no cabeçalho de `coupling.jl` como um afastamento deliberado do esboço tentativo de custo-igualmente-dividido de 13-RESEARCH.md, em favor do modelo teoricamente-mais-limpo em que a melhor resposta de cada distribuidor precifica apenas seu próprio investimento.

O `run_nash_probe` (`nash.jl`) instrumenta uma prática de robustez além do que a nota PSR especifica: como a diagonalização de Gauss-Seidel não tem garantia geral de unicidade/convergência (concern carregado de `STATE.md`), toda execução relatada é sondada através de múltiplas sementes (≥ 3) e ordens de varredura (≥ 2), reportando o **spread** observado entre execuções — nunca apresentando uma única execução como “o” equilíbrio.

Multiplicidade do equilíbrio (Fase 31, BILEV-06): a diagonalização de Gauss-Seidel converge para UM equilíbrio de Nash generalizado (GNE) do jogo de restrição compartilhada — não necessariamente o único. Um fixture com tetos de investimento INTERIORES (`x_inv_max=[1.0, 1.0]`, folga de 0.3 acima da necessidade de investimento total derivada `S_min=0.7`) expõe um continuum unidimensional genuíno de GNEs em (`x_inv_1`, `x_inv_2`) — toda divisão com `x_inv_1 + x_inv_2 = 0.7` (ambas dentro de `[0, 1.0]`) é um ponto fixo igualmente válido, com o MESMO par de fluxos (`z_1`, `z_2`) `approx (0.7, 0.7)` — regressão-testado em `test/test_planning_nash.jl` (“planning nash: interior-cap fixture ... exposes a genuine GNE continuum ...”). O mecanismo de sementes do `run_nash_probe` foi estendido para aceitar, além do `z0` bruto original, uma `(; z0, x_inv0) NamedTuple` (retrocompatível) — sondar esse continuum exige variar `x_inv0`, já que a derivação padrão do investimento seedado (`maximum(z0[j,:]) / corridor_cap`) tem folga ZERO por construção e por isso não consegue, por si só, expor essa multiplicidade. O fixture de teto de canto pré-existente (`x_inv_max=[0.3, 0.3]`) permanece como controle de equilíbrio ÚNICO (`spread ≈ 0`).

8.1 Equilíbrio variacional — `solve_variational_equilibrium`

Um equilíbrio VARIACIONAL (VE) é um GNE cujo multiplicador da restrição compartilhada (`capacity[t]`) é IDÊNTICO para todos os jogadores — a construção de equilíbrio normalizado de Rosen (1965) para jogos de restrição compartilhada convexos e separáveis (esta é exatamente a estru-

tura do jogo descrita acima: objetivos aditivamente separáveis, acoplados apenas por UMA restrição compartilhada). `solve_variational_equilibrium` (`nash.jl`) calcula um VE via UMA ÚNICA resolução monolítica de um modelo JuMP conjunto — o OPOSTO do laço de diagonalização de `run_nash!`: constrói N blocos independentes (`y_inv[i]`, `x_inv[i]`, `z[i,t]`, o oráculo de bem-estar completo de cada distribuidor) sobre o MESMO Model, com a linha de capacidade compartilhada escrita UMA ÚNICA vez (`capacity[t]: sum_i z[i,t] <= "corridor_cap" dot.c sum_i x_inv[i]`). O multiplicador $\pi_{\text{capacity}} = \text{dual}(\text{capacity})$ é, por construção, o MESMO vetor para todos os jogadores — não há nada para “igualar” entre eles.

O VE NÃO é único no fixture simétrico acima (correção da revisão de código da Fase 31, CR-02). O teorema de unicidade de Rosen exige concavidade diagonal estrita, que um jogo LINEAR em `x_inv` não tem. Com `c_inv = [1, 1]`, o objetivo conjunto e todas as restrições dependem de `x_inv` apenas através de `x_inv_1 + x_inv_2`; a face ótima do problema conjunto é o segmento inteiro de divisões, e TODO GNE do continuum tem o MESMO multiplicador ($\mu_i = 0.5$: com x_i interior, $c_{\text{inv}} = 2\mu_i$; no extremo $x_1 = 0$, a estacionariedade em z dá $\mu_1 = W'(0.7) - \lambda_0 - c_y - c_{\text{op}} = 0.5$). Ou seja, ali o conjunto de VEs É o conjunto de GNEs, e `solve_variational_equilibrium` devolve um ponto dessa face que depende do solver (o método de pontos interiores do Clarabel cai no centro analítico (0.35, 0.35)) — não há “seleção” alguma nesse fixture. O teste correspondente verifica apenas o que vale em toda a face (soma 0.7, $z \approx (0.7, 0.7)$, multiplicador 0.5, nenhum desvio unilateral lucrativo) — `test/test_planning_nash.jl` (“planning nash: solve_variational_equilibrium on the symmetric interior-cap fixture returns A point of the non-unique VE face ...”).

VE único num fixture assimétrico (CR-02). Com custo de investimento ASSIMÉTRICO, `c_inv = [1.0, 1.4]` (demais parâmetros iguais), o empate se desfaz. Pelas condições KKT de cada jogador, $z_i = 1.2 - \mu_i$ e $c_{\text{inv},i} \geq 2\mu_i$ (igualdade se $x_i > 0$): os GNEs formam um continuum com multiplicadores PRÓPRIOS de cada jogador (por exemplo $x_2 = 0$, $\mu_1 = 0.5$, $\mu_2 = p \in [0, 0.7]$, $x_1 = \frac{1.9-p}{2}$). O VE ($\mu_1 = \mu_2$) é ÚNICO: o problema conjunto compra capacidade apenas do jogador mais barato, $\mu = 0.5$, $z = (0.7, 0.7)$, `x_inv = (0.7, 0)`, `cost_per_distributor = [0.105, -0.595]`. Já a diagonalização de Gauss-Seidel a partir de `z0 = 0` (qualquer ordem) chega a OUTRO GNE, `x_inv = (0.35, 0.25)`, $z = (0.7, 0.5)$, com multiplicadores DESIGUAIS (0.5, 0.7) — aqui o VE seleciona genuinamente um ponto do continuum. Certificação: a divisão derivada à mão é verificada; para cada jogador, uma melhor resposta `solve_stackelberg!` (o outro fixado no VE) reproduz o z e o `cost_per_distributor` do VE, e o multiplicador PRÓPRIO desse jogador na linha compartilhada (lido da estacionariedade em z , via o dual do oráculo) coincide com $-\pi_{\text{capacity}}$ — `test/test_planning_nash.jl` (“planning nash: solve_variational_equilibrium selects the UNIQUE VE on an asymmetric-c_inv fixture ...”, CR-02).

Extensão inteira em $N > 1$ (Fase 31, BILEV-07): `run_nash!` aceita um kwarg `integer::Union{Nothing,NamedTuple}` (ex.: `integer = (; K=4,  $\alpha_{x\_lb}=0.0$ )`) que, quando fornecido, faz cada resposta ótima de cada distribuidor usar um `build_master_integer` FRESCO (nunca persistido entre respostas ótimas ou varreduras) em vez do `BendersMaster` contínuo — ver a seção anterior para o mapeamento completo de `build_master_integer/add_ll_cut!`. A diagonalização inteira detecta CICLOS sobre o estado binário conjunto EXATO (`Dict{Vector{Int},Int}`, nunca por tolerância) e falha ruidosamente se um ciclo genuíno ocorrer, em vez de relatar convergência falsa. Certificado em $N=2$, $K=4$ por força bruta (16 pontos de rede por jogador, nenhum desvio unilateral lucrativo) em `test/test_planning_nash_integer.jl`.

9 Resumo de equivalências

- A estrutura Stackelberg-via-Benders de uma distribuidora — mestre/líder (problema 4 ↔ `master.jl`) e seguidor (problema 2 ↔ `follower.jl`) — é **matematicamente idêntica**, com o mesmo tipo de corte de otimalidade (4f).

- O dual do acoplamento como custo marginal de reforço, π_s (2e) ↔ `follower.jl's` $\pi_s = \text{dual}(\text{f.coupling})$, é o mesmo objeto matemático.
- A diagonalização de Gauss-Seidel para o equilíbrio de Nash (problema 7) ↔ `nash.jl's` `run_nash!` é a mesma estrutura algorítmica, com o mesmo timing (write-back imediato, nunca Jacobi).
- O acoplamento N1↔N2 (1j)/(2d)/(2e)/(1g) ↔ as restrições nomeadas `coupling[t]/pin[t]` do repositório são a mesma relação física de acoplamento de fronteira, apenas implementadas com uma variável compartilhada em vez de uma igualdade entre duas variáveis.

10 Resumo de desvios deliberados

- **Sem índice de cenário:** o repositório não tem $s \in S$ nem a média $(\frac{1}{S}) \sum_s$ — cada iteração de Benders é um trial determinístico único, não uma expectativa estocástica.
- **Riqueza do oráculo além de $y_{\text{op},s}$ abstrato:** `subproblem.jl` reutiliza o SOCP de bem-estar COMPLETO da tese (dispositivos dos agregadores, rede branch-flow, eq. 3.31–3.45) como a “operação de N2”, em vez do balanço linear abstrato $F_y y_{\text{op},s} = d_{y,s}$ da nota — um reforço, não uma equivalência simples, e por isso o repositório precisa de um segundo epígrafe de Benders (α_{op}) que a nota PSR não precisa.
- **Minimalismo do seguidor:** `follower.jl` é uma LP deliberadamente minimalista de UM ÚNICO corredor escalar, sem a rede interna de N1 (balanço de demanda (1c)/(2c)) que a nota PSR postula genericamente.
- **Convenção de sinal de z :** `master.jl's` $z[t]$ é limitado em $[0, y_{\text{inv}}]$ — um fluxo de importação fisicamente não negativo — em vez de um fluxo com sinal livre.
- **Alocação de custo de investimento compartilhado (Nash):** `coupling.jl` resolve a ambiguidade de alocação de custo do problema (6)/(7) da nota com propriedade de investimento POR distribuidor, uma decisão documentada explicitamente (ver Seção 8).
- **Extensão inteira (problemas 8-9), mecanismo e alvo distintos da nota:** implementada desde a Fase 24 (v3.0), estendida ao equilíbrio de Nash $N > 1$ na Fase 31 (v4.0) — `master_integer.jl's` `build_master_integer/add_ll_cut!` integerizam o investimento do LÍDER (y_{inv} , N2) via expansão binária `b[1:K]`, impondo o recourse exato por cortes L-shaped inteiros de Laporte-Louveaux — não o investimento/fluxo do SEGUIDOR (N1) via relaxação Lagrangeana de restrições de cópia que a nota PSR usa (ver Seção 8 e a tabela de extensão inteira acima para o mapeamento completo).

11 Referências

Referências:

Palacios, J. P. *Tese de Doutorado*, UNSJ/CONICET, 2022, Capítulo 4 (camada de planejamento).

Nota PSR: “Reforços interconexões N1-N2 através de equilíbrios de Stackelberg e Nash”, documento interno, Junho 2026 (docs/references/Expansão-N1-N2-EquilíbriosStackelberg-Nash-v2 (1).pdf, 9 pp.) — lida diretamente para este documento. `.planning/research/THEORY-papers.md` — dígest cruzado das duas fontes (Paper 1 IET-2019 + nota PSR); ver Seção 3 para uma correção pontual encontrada nesse dígest.

Zou, J., Ahmed, S., Sun, X.A. (2019). “Stochastic dual dynamic integer programming.” *Mathematical Programming* 175, 461–502.
Bansal, A., Küçükyavuz, S. (2025). “Integer L-shaped and Lagrangian cuts revisited: a unified perspective.” *Operations Research Letters*.

Implementação: `src/planning/master.jl`, `src/planning/follower.jl`, `src/planning/subproblem.jl`, `src/planning/benders.jl`, `src/planning/coupling.jl`, `src/planning/nash.jl` no projeto TSO-DSO.

Documento irmão (narrativa da implementação): `modelo_stackelberg_dso_unico.typ` — este documento complementa-o com o mapeamento termo-a-termo contra a fonte PSR; consulte-o para a narrativa livre do laço de Benders e do resultado.